

Unit3: Data Link Layer

The data link layer is the second layer in the OSI (Open Systems Interconnection) model of computer networking. It is responsible for providing reliable communication between adjacent network nodes over a physical link, such as a local area network (LAN) or a wide area network (WAN).

In the OSI model, the data link layer is the 2nd layer from the bottom.

It is responsible for transmitting frames from one node to next node.

The main responsibility of the Data Link Layer is to transfer the datagram across an individual link.

An important characteristic of a Data Link Layer is that datagram can be handled by different link layer protocols on different links in a path.

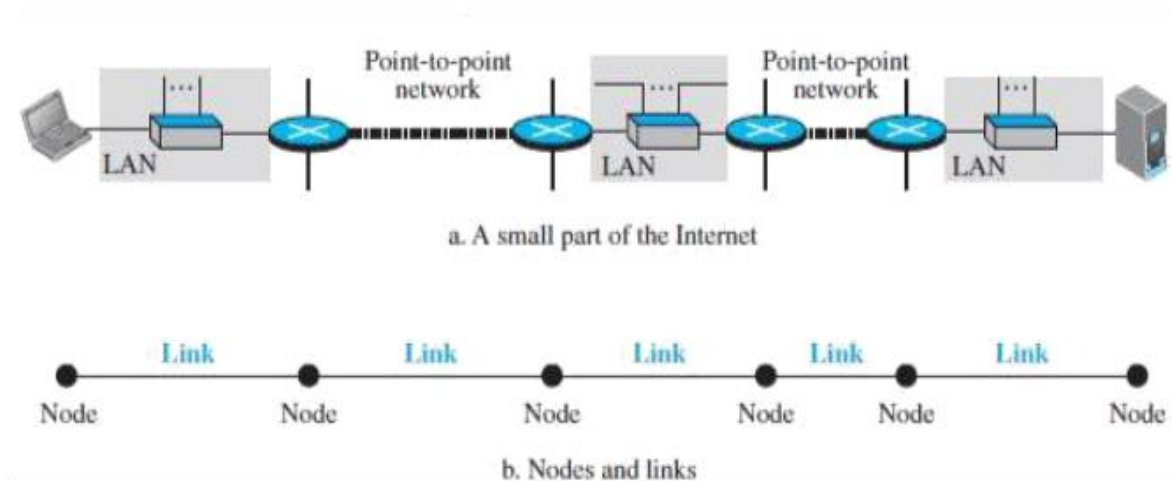
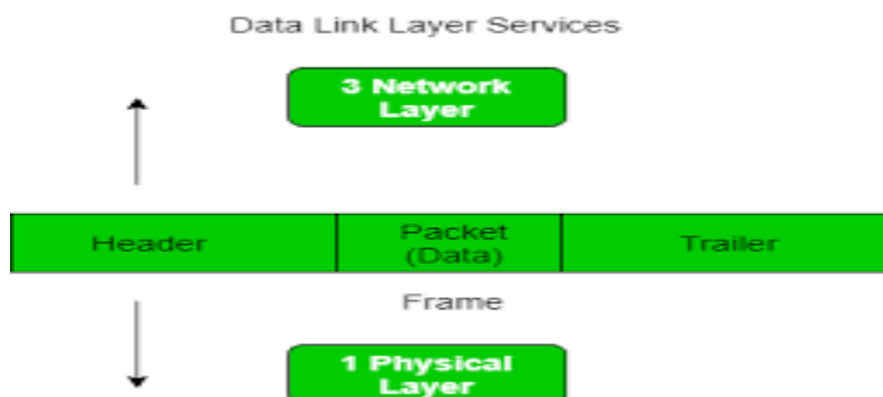


FIGURE 2.1: NODES AND LINKS



The data link layer divides data into frames, adds a header and a trailer to each frame, and ensures that the frames are transmitted correctly over the physical link. It also handles error detection and correction, flow control, and access control to the physical media.

1. Header – Why Used?

The **Header** is added at the **starting** of the frame.

1. **Source Address**
→ Who is sending the data?
2. **Destination Address**
→ Who should receive the data?
3. **Control Information**
→ Frame number (for ordering)
→ Type of frame (Data)
4. **Protocol Information**
→ Which upper-layer protocol is used?

2. Trailer – Why Used?

The **Trailer** is added at the **end** of the frame.

1. **Error Detection**
 - CRC (Cyclic Redundancy Check)
 - Checksum
 - Parity bits
2. **Helps receiver check:**
→ Data is correct or corrupted?

It is responsible for **node-to-node delivery** of data.

It converts **packets into frames**.

Data Link Layer Design Issues:

There are several design issues that must be considered when designing the data link layer. Here are some of the key ones:

1. **Framing:**

The data link layer must break up data into manageable units known as frames. The frame should be of a fixed or variable length and should include a header and a trailer that identify the frame boundaries.

2. **Error Detection and Correction:**

The data link layer must detect and correct any errors that may occur during transmission of frames. Common techniques for error detection and correction include checksums, cyclic redundancy check (CRC), and forward error correction (FEC).

3. **Flow Control:**

The data link layer must regulate the flow of data to prevent overwhelming the receiver with too much data. Flow control can be implemented using techniques such as buffering, windowing, and congestion control.

4. **Medium Access Control:**

The data link layer must determine how devices access the medium for transmission. This can be done through techniques such as contention-based access (such as CSMA/CD used in Ethernet) or reservation-based access (such as Token Ring).

5. **Addressing:**

The data link layer must provide addressing mechanisms to uniquely identify each device on the network. This can be done using either physical addresses (such as MAC addresses) or logical addresses (such as IP addresses).

6. **Protocol Implementation:**

The data link layer must implement the appropriate protocol(s) to support the transmission of data over the physical medium. Examples of data link layer protocols include Ethernet, HDLC, and PPP

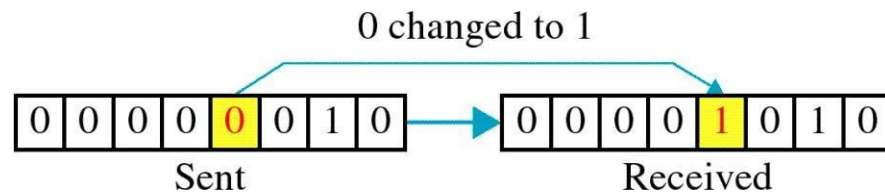
Error detection and correction

In data communication, an **error** means the received data is **different** from the transmitted data due to noise, interference, or signal problems.

1. **Single-Bit Error:-**

The term single –bit error means that only one bit of given data unit is changed from 1 to 0 or 0 to 1.

When **only one bit** of a data unit changes from $0 \rightarrow 1$ or $1 \rightarrow 0$.



2. Burst Error :-

The term burst error means 2 or more than 2 bits in data unit have changed from 0 to 1 or 1 to 0.

When **two or more bits** in a data unit are changed.

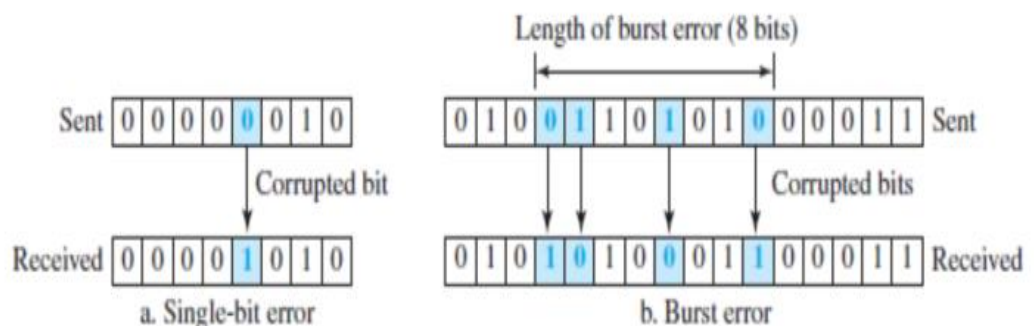
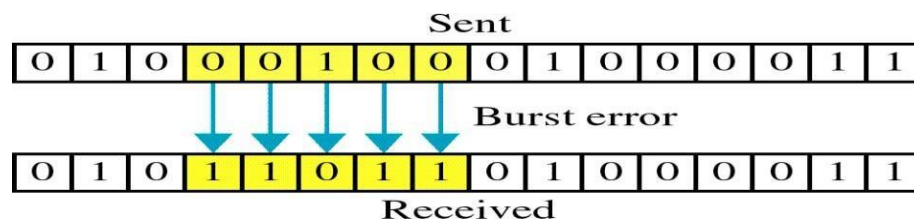


FIGURE 2.3: SINGLE-BIT AND BURST ERROR

3. Hamming Distance

The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits. The Hamming distance can easily be found if we apply the XOR operation on the two words and count the number of 1s in the result. Note that the Hamming distance is a value greater than zero.

The number of bit positions in which two binary numbers are different. It is used in Error Detection and Error Correction in the Data Link Layer.

Hamming Distance between two binary strings of equal length is the number of positions at which the corresponding bits are different.

- The **Hamming distance** between two strings of equal length is **just the number of positions where the symbols are different**.
- Think of it as “how many changes you need to make to turn one string into the other.”

Step-by-Step Example1

Take two binary strings:

String 1: 10101

String 2: 10011

Compare them **position by position**:

Position	String 1	String 2	Same or Different?
1	1	1	Same
2	0	0	Same
3	1	0	Different
4	0	1	Different
5	1	1	Same

- Count the differences → 2 positions differ.
- So, **Hamming distance = 2**

Example 2: Letters (non-binary)

Compare two 4-character strings:

$$s_1 = \text{"CODE"}, s_2 = \text{"CAME"}$$

Compare each character:

Position	1	2	3	4
s1	C	O	D	E
s2	C	A	M	E

- Position 2: O vs A → difference
- Position 3: D vs M → difference

Hamming distance = 2

Example 3:

Compare 4-bit binary strings:

$$A = 1101, B = 1000$$

Compare two 5-character strings:

$$S_1 = \text{"HELLO"}, S_2 = \text{"CELLO"}$$

Why It Is Important?

1. **Error Detection** – Helps detect if data was corrupted during transmission.
2. **Error Correction** – Determines how many errors can be fixed automatically.
3. **Data Integrity** – Ensures transmitted or stored data matches the original.
4. **Measuring Similarity** – Compares binary strings, DNA sequences, or patterns.
5. **Designing Codes** – Helps in creating codes with good error-handling ability (like Hamming codes).
6. **Digital Communication Reliability** – Keeps computer networks, storage devices, and communication systems reliable.
7. **AI & Machine Learning** – Measures differences between feature vectors or binary-encoded data.

Minimum Hamming Distance:

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

The **minimum Hamming distance** of a code is the **smallest number of bits that differ between any two codewords** in the code.

It tells us **how many errors the code can detect or correct**.

Example

Suppose we have 3-bit codewords:

$$C_1 = 000, C_2 = 011, C_3 = 101, C_4 = 110$$

Compare all pairs of codewords:

Pair	Hamming distance
C_1, C_2	2 $C_1 = 000$ $C_2 = 011 \rightarrow 2$
C_1, C_3	2
C_1, C_4	2
C_2, C_3	2
C_2, C_4	2
C_3, C_4	2

- Minimum Hamming distance $d_{\min} = 2$

Example (from our 2-bit code)

Codewords:

$$C_1 = 00, C_2 = 01, C_3 = 11$$

Hamming distances:

Pair	Distance
C_1, C_2	1
C_1, C_3	2
C_2, C_3	1

- Smallest distance = 1
- Pairs with distance 1: C_1, C_2 and C_2, C_3
- The minimum Hamming distance $d_{\min} = 1$

4-bit code example

Codewords	Pair	Hamming Distance
$C_1 = 0000$	C_1, C_2	2
$C_2 = 0011$	C_1, C_3	3
$C_3 = 1100$	C_1, C_4	2
$C_4 = 1111$	C_2, C_3	2
	C_2, C_4	2
	C_3, C_4	2

Minimum Hamming distance $d_{\min} = 2$

Example 1: 3-bit code

Codewords:

$$C_1 = 000, C_2 = 011, C_3 = 101, C_4 = 110$$

Example 2: 4-bit code

Codewords:

$$C_1 = 0000, C_2 = 0011, C_3 = 1100, C_4 = 1111$$

Error detection methods:-

There are 3 types of detection methods.

1. Parity check
2. Cyclic Redundancy check
3. Checksum Parity check:-

1-Simple parity check:-

Parity check is a simple error detection technique used in digital communication to detect errors in transmitted data.

It works by adding one extra bit, called a parity bit, to a group of data bits.

The parity bit makes the total number of 1s either:

- Even → Even Parity
- Odd → Odd Parity

If the received data does not match the expected parity, an error is detected.

Types of Parity

Even Parity

The total number of 1s (including the parity bit) must be **even**.

Odd Parity

The total number of 1s (including the parity bit) must be **odd**.

Working of Parity Check

1. The sender counts the number of 1s in the data.
2. A parity bit is added according to the chosen parity type.
3. The receiver counts the number of 1s again.
4. If the parity rule is violated, an error is detected.

Example (Even Parity)

Suppose the data is:

1011010

Step 1: Count number of 1s

$1+0+1+1+0+1+0 = 4$ ones (Even)

Since we are using **even parity**, the parity bit = 0

Transmitted data:

10110100

Total number of 1s = 4 (Even)

If an error occurs during transmission:

Received data:

10111100

Number of 1s = 5 (Odd)

Since even parity was used and the count is now odd → **Error is detected**

Advantages

- Simple and easy to implement
- Low cost
- Requires only one extra bit

Limitations

- Detects only **single-bit errors**
 - Cannot correct errors
-

2-Another Example of Parity Check (Odd Parity)

Given Data:

1101001

Step 1:

Count the number of 1s

$1 + 1 + 0 + 1 + 0 + 0 + 1 = 4$ ones (Even)

Step 2:

Use Odd Parity

Since we are using odd parity, the total number of 1s must be odd.

Currently, there are 4 ones (even).

So, we add parity bit = 1

Transmitted Data:

11010011

Now total number of 1s = 5 (Odd)

Suppose an Error Occurs During Transmission

Received Data:

11000011

Count number of 1s:

$1 + 1 + 0 + 0 + 0 + 0 + 1 + 1 = 4$ ones (Even)

But we used odd parity, so the total should be odd.

Parity rule violated

Error detected

Conclusion

In odd parity:

- If total 1s are odd → No error
 - If total 1s are even → Error detected
-

Cyclic Redundancy Check, Checksum:-

What is Cyclic Redundancy Check (CRC)?

Cyclic Redundancy Check (CRC) is an error-detecting technique used in digital networks and storage devices to detect accidental changes to raw data during transmission or storage.

It is widely used in:

- Ethernet networks
- USB communication
- Hard drives
- Data communication protocols like HDLC

These are basic CRC terms.

Dataword is the original data that we want to send.

It contains only the actual information bits.

Example:

Dataword = 1101

This is the message before adding CRC.

Generator (Generator Polynomial)

Generator is a predefined binary pattern used to calculate the CRC.

It is also called **generator polynomial**.

Example:

Generator = 1011

This represents polynomial: $x^3 + x + 1$

Both sender and receiver must use the **same generator**.

Codeword is the final transmitted data.

It contains: Codeword = Dataword + CRC bits

So, it is longer than the original data.

CRC treats data as a **binary polynomial** and performs **modulo-2 division** to generate a short fixed-length check value called the CRC code or checksum.

What is a Binary Polynomial?

In CRC, binary data (0s and 1s) is treated like a **polynomial expression**.

Example:

Binary number:

1101

This is treated as a polynomial:

$$1x^3 + 1x^2 + 0x^1 + 1x^0$$

Which means:

$$x^3 + x^2 + 1$$

Each bit represents a coefficient (0 or 1).

The position represents the power of **x**.

So:

Binary	Polynomial
1011	$x^3 + x + 1$
1001	$x^3 + 1$
1101	$x^3 + x^2 + 1$

We don't actually use **x** in calculation — it's just a mathematical way to represent the bits.

What is Modulo-2 Division?

Modulo-2 division is **binary division without carry and without borrow**.

Instead of normal subtraction, we use:

XOR operation ($\oplus$)

A	B	$A \oplus B$
0	0	0
1	1	0
1	0	1
0	1	1

This is exactly like subtraction in modulo-2 arithmetic.

Why “Modulo-2”?

Because:

- Only two numbers exist: 0 and 1
- All calculations wrap around mod 2
- $1 + 1 = 0$ (not 2)

Example:

Normal math:

$$1 + 1 = 2$$

Modulo-2 math:

$$1 + 1 = 0$$

Simple Example of Modulo-2 Division Let's divide:

Data: 1101

Generator: 1011

Step 1: Compare first 4 bits

1101

1011

0110 (XOR result)

How CRC Works

Sender Side

1. Choose a generator polynomial (e.g., 1011).
2. Append zeros to the data equal to the degree of the generator polynomial.

The **degree** of a polynomial is the **highest power of x**.

Example: Generator = 1011

This represents: $x^3 + x + 1$

Highest power = 3 So, degree = 3. 1101 → 1101000

Number of zeros = (Number of bits in generator – 1)

Why Do We Add Zeros?

We add zeros to:

Make space for the CRC remainder

Allow division to produce the remainder

Later replace those zeros with the actual CRC bits

After division, the remainder (3 bits here) replaces those 3 zeros.

3. Perform binary division (XOR division).
4. The remainder becomes the CRC bits.
5. Append the CRC bits to the original data.
6. Transmit the new combined message.

Receiver Side

1. Receive the transmitted data (data + CRC).
2. Divide it again using the same generator polynomial.
3. If the remainder is:
 - **0 → No error detected**
 - **Non-zero → Error detected**

CRC Example (Binary Example)

Given:

- Data = 1101
- Generator = 1011 (degree 3)

Step 1: Append zeros

Add 3 zeros (because generator degree = 3)

1101 → 1101000

Step 2: Perform Modulo-2 Division (XOR Division)

After division, assume remainder = 001

Step 3: Append Remainder

Transmitted message:

1101 001

Receiver Check:

If division result = 0 → Data accepted

If not → Error detected

Why CRC is Powerful

CRC can detect:

- All single-bit errors
- Most double-bit errors
- Odd number of bit errors
- Burst errors (depending on polynomial used)

For example, CRC-32 used in Ethernet is extremely reliable for detecting transmission errors.

Advantages of CRC

Very strong error detection capability

Simple hardware implementation (using shift registers)

Efficient for high-speed networks

Detects burst errors effectively

Widely standardized

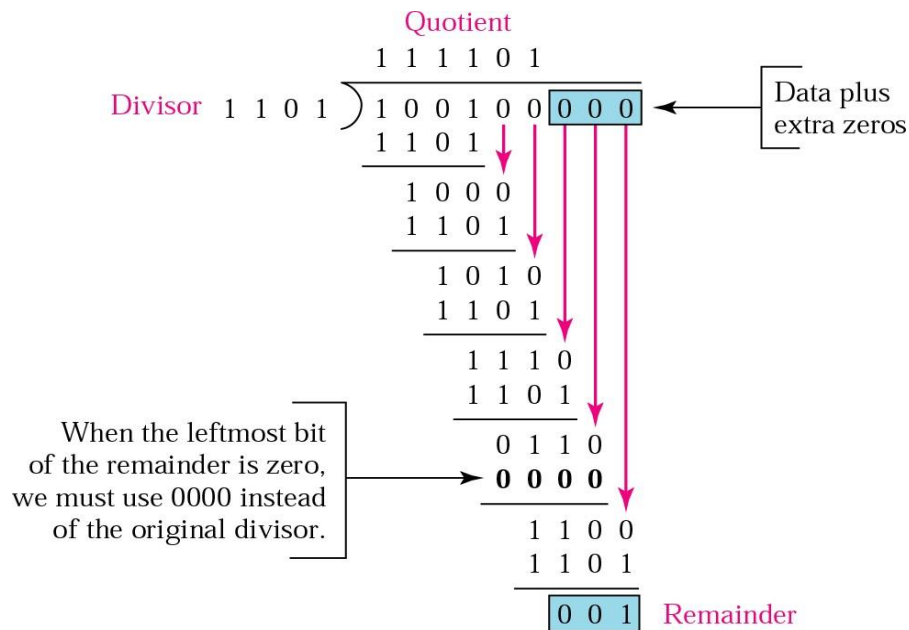
Disadvantages of CRC

Cannot correct errors (only detects)

Slightly more complex than simple parity check

Requires agreed generator polynomial

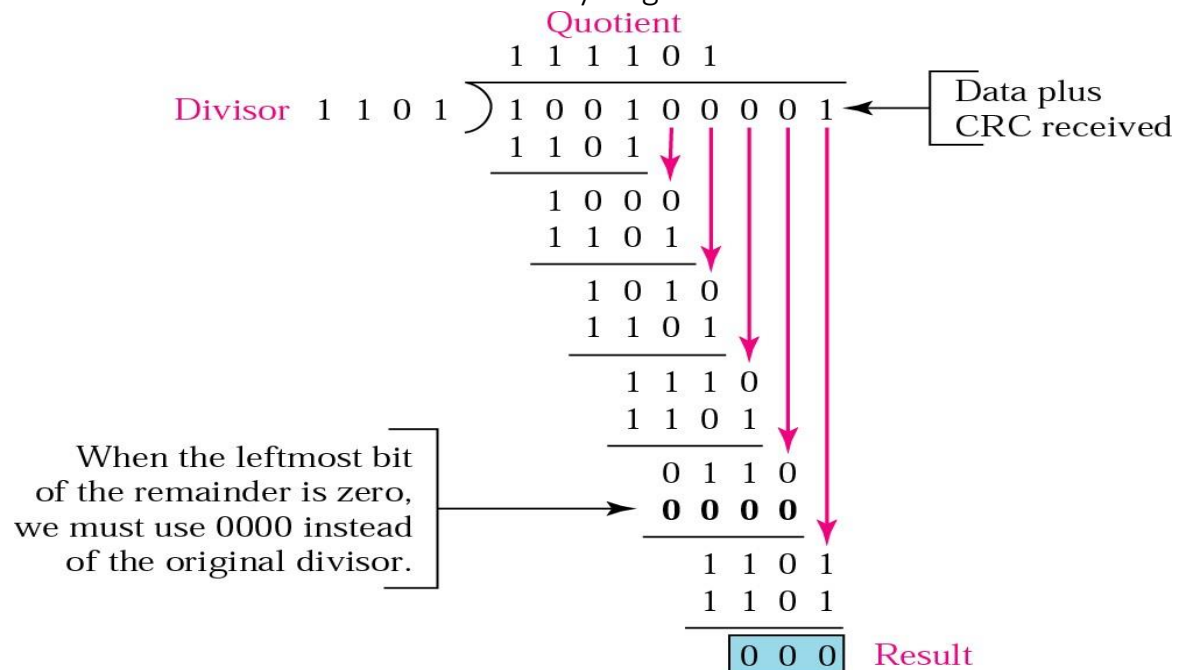
Not suitable for cryptographic security



Binary division by CRC generator

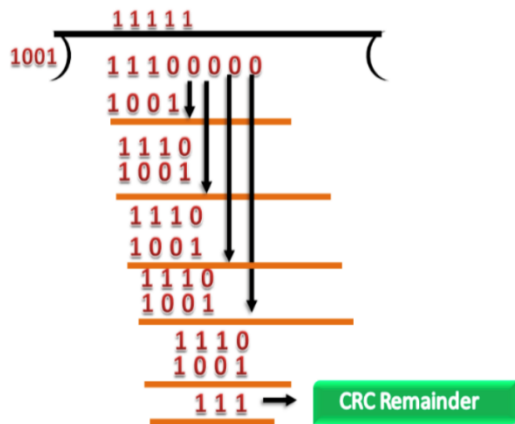
The CRC checker –

A CRC checker functions exactly as generator does

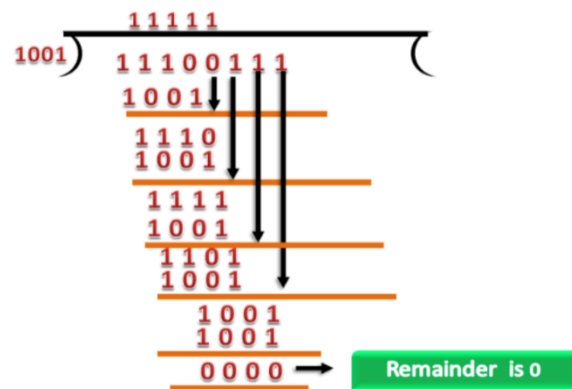


Explanation of CRC Generator & Checker with Example:

CRC Generator - Sender



CRC Checker - Receiver



CHECKSUM

Checksum is an error-detection technique in which the sender adds the data bits together and sends the computed value (called checksum) along with the data to detect errors during transmission.

Why Do We Need Checksum?

During data transmission:

- Noise can change bits ($0 \rightarrow 1$ or $1 \rightarrow 0$)
- Data may get corrupted

Checksum helps to:

Detect errors in transmitted data

Ensure data integrity

Improve communication reliability

It is commonly used in:

- Transmission Control Protocol (TCP)
- User Datagram Protocol (UDP)
- IP communication

How Checksum Works

Sender Side

1. Divide data into equal-sized blocks (usually 8 or 16 bits).
2. Add all blocks using binary addition.
3. If overflow occurs, add carry back to result.
4. Take 1's complement of the final sum.
5. Send:

Data + Checksum

Receiver Side

1. Add all received blocks (including checksum).
 2. If result = all 1s → No error.
 3. If result ≠ all 1s → Error detected.
-

Example

Suppose 4-bit blocks:

Data:

1010

0101

Step 1: Add

1010

+0101

1111

Step 2: Take 1's complement

0000 → Checksum

Step 3: Transmit

1010 0101 0000

At receiver:

If total = 1111 → No error

Otherwise → Error detected

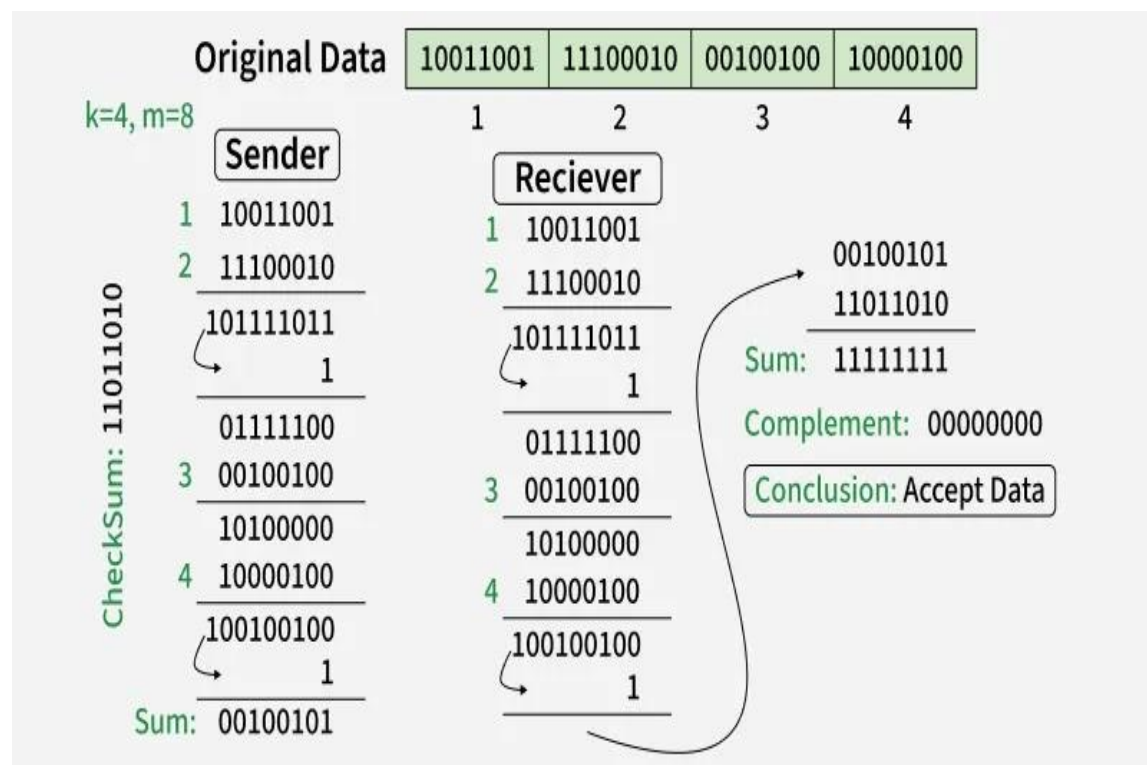
Advantages of Checksum

Simple to implement
Low computational cost
Faster than CRC
Suitable for small systems

Disadvantages of Checksum

Cannot detect all errors
Weak at detecting burst errors
Less reliable than CRC
Not secure against intentional modification.

- **k** represents the **number of segments** (or blocks) the data is divided into.
- **m** represents the **number of bits** in each of those segments.



Sender's End	Receiver's End
Frame 1: 11001100 Frame 2: + 10101010 Partial Sum: 1 01110110 + 1 01110111 Frame 3: + 11110000 Partial Sum: 1 01100111 + 1 01101000 Frame 4: + 11000011 Partial Sum: 1 00101011 + 1 Sum: 00101100 Checksum: 11010011	Frame 1: 11001100 Frame 2: + 10101010 Partial Sum: 1 01110110 + 1 01110111 Frame 3: + 11110000 Partial Sum: 1 01100111 + 1 01101000 Frame 4: + 11000011 Partial Sum: 1 00101011 + 1 Sum: 00101100 Checksum: 11010011 Sum: 11111111 Complement: 00000000 Hence accept frames.

The Hamming code algorithm :-

The Hamming code algorithm is an error-correcting code that can detect and correct single-bit errors in binary data. It was developed by Richard Hamming in the late 1940s.

The Hamming code works by adding extra parity bits to the data being transmitted. These parity bits are calculated based on the position of the bit in the data stream, and are used to check whether any errors have occurred during transmission.

Here are the steps for encoding data using the Hamming code algorithm:

1. Determine the number of parity bits

The number of parity bits (r) required for m data bits is calculated using the formula:

$$2^r \geq m + r + 1$$

where

m = number of data bits

r = number of parity bits

2. Place parity bits in the data

The parity bits are inserted at positions that are **powers of 2**.

Example positions:

1, 2, 4, 8, 16 ...

The remaining positions are filled with **data bits**.

3. Calculate parity bits

Each parity bit checks certain positions in the data.

The value of the parity bit is determined by **counting the number of 1s** in the corresponding positions.

- If the number of 1s is **even**, the parity bit is **0**.
 - If the number of 1s is **odd**, the parity bit is **1**.
-

4. Insert parity bits into the data

After calculating the parity bits, they are inserted into their respective positions in the data stream to form the **final transmitted Hamming code**.

Example data: **1101**

Position	1	2	3	4	5	6	7
Type	P1	P2	D1	P4	D2	D3	D4
Bits	P1	P2	1	P4	1	0	1

Hamming Code Diagram

Bit Position : 1 2 3 4 5 6 7

Bit Type : P1 P2 D1 P4 D2 D3 D4

Data/Parity : ? ? 1 ? 1 0 1

Parity checking groups:

P1 → checks positions : 1,3,5,7

P2 → checks positions : 2,3,6,7

P4 → checks positions : 4,5,6,7

After calculating parity bits, the final transmitted code becomes:

Final Hamming Code

1 0 1 0 1 0 1

5-Error Detection at Receiver

1. The receiver recalculates parity bits.
 2. If parity values match → **No error**.
 3. If they do not match → **Error is detected**.
 4. The position of the error is identified and the incorrect bit is **flipped to correct it**.
-

6-Error Detection and Correction

At the receiver side, the parity bits are recalculated and compared with the received parity bits.

- If the parity bits match → **No error**
- If they do not match → **An error is detected**

The position of the error can be determined using the incorrect parity bits. The receiver then **flips the incorrect bit to correct the error**.

Conclusion

The Hamming code algorithm is widely used in **communication systems and computer memory** to ensure **data accuracy and reliability**. It can **detect and correct single-bit errors** efficiently.

Framing in Data Link Layer

In Computer Networking, Framing is a function of the Data Link Layer that divides the continuous stream of bits received from the Physical Layer into manageable units called frames.

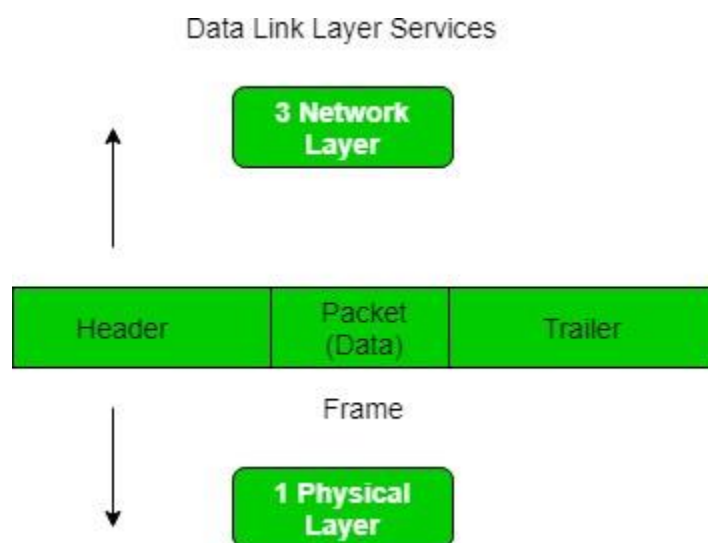
A frame is a structured block of data that contains the data to be transmitted along with control information such as source address, destination address, and error-checking bits.

Framing is the process of encapsulating network layer packets into frames so they can be transmitted reliably over a physical communication channel.

Need for Framing

Framing is important because it:

- Identifies the beginning and end of data.
- Helps in error detection.
- Enables flow control and synchronization between sender and receiver.
- Allows the receiver to separate different data blocks.



At the data link layer, it extracts the message from the sender and provides it to the receiver by providing the sender's and receiver's addresses. The advantage of using frames is that data is broken up into recoverable chunks that can easily be checked for corruption.

The process of dividing the data into frames and reassembling it is transparent to the user and is handled by the data link layer.

Framing is an important aspect of data link layer protocol design because it allows the transmission of data to be organized and controlled, ensuring that the data is delivered accurately and efficiently.

Problems in Framing

1. Frame Synchronization Problem

The receiver must know where a frame starts and where it ends in a data stream. This process is called frame synchronization.

If synchronization is lost due to transmission errors, the receiver cannot correctly identify the boundaries of frames. As a result, the receiver may read the data incorrectly, which can lead to wrong interpretation of the information.

Example:

Data Stream: 101110101010

If the frame boundaries are not known, the receiver may divide the data incorrectly.

- **Correct reading:** 1011 | 1010 | 1010
- **Incorrect reading:** 10 | 1110 | 1010 | 10

Because of this, the data may be interpreted incorrectly.

In short:

The Frame Synchronization Problem occurs when the receiver cannot correctly identify the start and end of frames in a data stream.

2. Corruption of Length Field

The Length Field in a frame tells the receiver how many bytes or bits of data are present in that frame.

If the length field gets corrupted due to transmission errors, the receiver may read the wrong frame size. This can cause the receiver to read too much data or too little data, leading to incorrect frame interpretation.

Example:

If the actual length of a frame is 10 bytes, but due to an error the length field changes to 15 bytes, the receiver will try to read extra data, which may belong to the next frame.

3. Flag Byte Appearing in Data

In some communication methods, a special flag byte is used to mark the start and end of a frame.

Example flag: 01111110

Problem happens when the same pattern appears inside the data.

Then the receiver gets confused and thinks the frame has ended, even though it is just normal data.

Example:

Flag = 01111110

Frame sent:

01111110 | 10101101111110 | 1101 | 01111110

Here 01111110 inside the data may look like a frame end to the receiver.

So the receiver may cut the frame at the wrong place and read the data incorrectly.

4. Bit Errors During Transmission

During data transmission, sometimes bits change because of noise or other disturbances in the communication channel. This problem is called a Bit Error.

It means a 0 may change to 1 or a 1 may change to 0 while the data is being transmitted.

Because of this, the receiver gets incorrect data.

Example:

Sent Data:

1011001

Received Data (after error):

1001001

Here one bit has changed, which causes the data to become incorrect.

In short:

A Bit Error occurs when a bit changes from 0 to 1 or from 1 to 0 during transmission.

5. Loss or Duplication of Frames

Loss or Duplication of Frames is a problem that can occur during data transmission.

- Loss of Frames: A frame sent by the sender does not reach the receiver.
- Duplication of Frames: The same frame is received more than once by the receiver.

This usually happens because of network errors or retransmission of frames.

Example:

Sender sends:

Frame1, Frame2, Frame3

- Frame Loss: Receiver gets → Frame1, Frame3 (Frame2 is missing)
- Frame Duplication: Receiver gets → Frame1, Frame2, Frame2, Frame3

In short:

Loss or Duplication of Frames means a frame may be missing or received multiple times during transmission.

Types of framing

In Computer Networking, framing in the Data Link Layer is often broadly classified into two main types based on frame size.

1. Fixed Size Framing

In fixed size framing, all frames have the same length.

Fixed Size Framing is a framing method in which all frames transmitted over the network have the same predetermined size. The sender and receiver both know the frame size in advance, so no special start or end markers are required.

- Each frame contains a fixed number of bytes.
- No need to indicate the start and end of the frame.
- The receiver simply reads a fixed number of bits each time.

Characteristics

1. Constant Frame Length:

- Each frame has the **same number of bytes or bits**.
- Example: If the frame size is 8 bytes, every frame contains **exactly 8 bytes**.

2. Simple Processing:

- The receiver **does not need to check for start/end markers**.
- It simply reads the next **fixed number of bits** as a frame.

3. No Need for Extra Bits:

- Unlike variable size frames, there's no need for **length fields** or **flag bytes**.

Working Example

Suppose the frame size is **8 bits**. The bit stream received is:

10101011 11001100 10011010 01100110

The receiver divides the stream into **fixed 8-bit frames** automatically:

1	1	1	0
0	1	0	1
1	0	0	1
0	0	1	0
1	1	1	0
0	1	0	1
1	0	1	1
1	0	0	0

- Each frame is 8 bits long.
 - The receiver reads one frame at a time without worrying about start or end markers.
-

2. Variable Size Framing

In variable size framing, frames can have different lengths depending on the data.

So, the sender must mark the beginning and end of the frame.

Variable Size Framing is a framing method in which frames can have different lengths depending on the amount of data to be transmitted.

Unlike fixed size frames, the sender must include information to indicate where each frame starts and ends.

Definition:

In Fixed Size Framing, **every frame has the same length**, regardless of how much data is actually being sent.

How it works:

- The sender divides the data stream into frames of **equal size**, say 100 bytes each.
- The receiver **knows in advance** that every frame is 100 bytes, so it can read the data easily without checking for start/end markers.

Advantages:

1. **Simple:** Easy to implement in hardware and software.
2. **Fast:** Receiver can immediately know where the next frame starts.
3. **Easy to detect boundaries:** No need for length fields or flags.

Disadvantages:

1. **Wastes space:** If data is less than frame size, extra space must be padded.
2. **Not flexible:** Cannot handle variable-length data efficiently.

Example:

- Frame size = 100 bytes
- If data = 250 bytes, it will be divided as:
 - Frame1 = 100 bytes
 - Frame2 = 100 bytes
 - Frame3 = 50 bytes + 50 bytes padding

Key Idea: Fixed size is **fast and predictable**, but may waste space.

2. Variable Size Framing

Definition:

In Variable Size Framing, frame length can change depending on the data.

How it works:

- Each frame can have a different number of bytes.
- The frame boundaries are marked using:
 1. Length Field: Tells receiver how long the frame is.
 2. Flags / Special Pattern: Marks start and end of frame.

Advantages:

1. Flexible: Can handle frames of different sizes.
2. Efficient: Less wasted space; no need for padding.

Disadvantages:

1. Complex: Receiver must read length field or check flags.
2. Slower: Extra processing needed to detect frame boundaries.

Example:

- Frame1 = 50 bytes
- Frame2 = 120 bytes
- Frame3 = 80 bytes

The receiver reads the **length field** or looks for **flags** to know exactly how many bytes belong to each frame.

Key Idea: Variable size framing is **efficient and flexible**, but **slightly more complex** to implement.

Methods used:

- Character count
- Byte stuffing
- Bit stuffing
- Flag bytes

Characteristics

1. Flexible Frame Length:
 - Frame size can vary according to data.
 - Useful when data units are not uniform.
2. Requires Delimiters or Length Information:
 - Start and end of a frame must be explicitly identified.
 - Can be done using:
 - Length field (Character Count Method)
 - Flag bytes (Byte Stuffing)
 - Bit Stuffing
 - Special signal patterns (Physical Layer Violations)
3. Error Handling Needed:
 - Receiver must detect frame boundaries carefully to avoid loss of synchronization.

Advantages

1. Efficient bandwidth usage:
 - No unused bits as in fixed size frames.
 2. Flexible:
 - Can handle variable-length data packets.
 3. Better for bursty traffic:
 - Suitable for networks where data lengths vary frequently.
-

Disadvantages

1. Complex implementation:
 - Receiver must detect frame boundaries correctly.
2. Synchronization overhead:
 - Requires flags, length fields, or stuffing mechanisms.
3. Error propagation risk:
 - If a frame delimiter is corrupted, the receiver may lose alignment.

Methods Used in Variable Size Framing

Variable size frames need explicit ways to mark the start and end of a frame. The following methods are commonly used:

1. Character Count Method

- Idea: The first byte or field of the frame specifies the number of characters (bytes) in the frame.
- How it works:
 - The receiver reads the length field first.
 - Then it reads the specified number of characters as the frame.

Example:

Length = 5

Data = ABCDE

- Total frame = Length + Data = 6 bytes

Advantage: Simple to implement.

Disadvantage: If the length field is corrupted, frame synchronization is lost.

2. Byte Stuffing (Character Stuffing)

- Idea: Use special flag characters to mark the start and end of a frame.
- If the flag appears in the data, it is escaped using an ESC character.

Example:

FLAG DATA FLAG

- Data: A FLAG B → Transmitted as A ESC FLAG B

Advantage: Prevents misinterpretation of data as frame boundaries.

Disadvantage: Slight overhead due to ESC characters.

3. Bit Stuffing

- Idea: Used in bit-oriented protocols like HDLC.
- Rule: After five consecutive 1s in the data, a 0 is inserted automatically.
- Receiver removes the extra 0 to retrieve original data.

Example:

Original Data: 1111101111

After Stuffing: 11111001111

Advantage: Avoids confusion with frame boundary flags (01111110).

Disadvantage: Requires bit-level processing.

4. Flag Bytes (Start/End Flag)

- Idea: Special unique bit pattern (e.g., 01111110 in HDLC) is used to mark the start and end of a frame.
- All frames are enclosed by this flag pattern.

Example:

01111110 DATA 01111110

Advantage: Clear and easy to detect frame boundaries.

Disadvantage: If the flag pattern appears in data, bit stuffing is **needed**.

Flow Control:

Flow Control means controlling the rate of data transmission between a sender and a receiver so that the receiver is not overwhelmed by too much data at once.

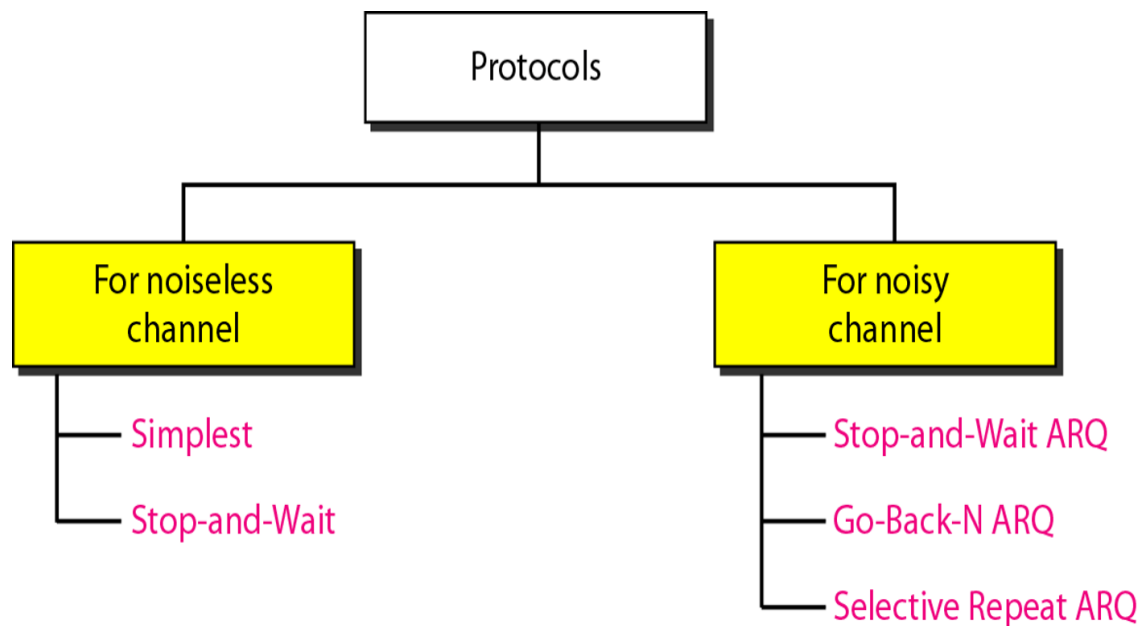
Definition

Flow control is a technique used in data communication to regulate the speed of data transfer between sender and receiver to ensure the receiver can process the data properly.

Example

If a sender sends data very fast and the receiver processes data slowly, the receiver's buffer may overflow and data can be lost.

Flow control prevents this problem by controlling how much data the sender can send.



1-Noiseless Channel:

A **noiseless channel** is a theoretical or ideal communication channel in which data transmitted from the sender to the receiver arrives **without any errors, loss, or corruption**.

Key Characteristics:

- **No errors occur** during transmission.
- Data frames arrive exactly as sent.
- There is **no need for error detection or correction**.
- Focus is only on **flow control** (managing the rate of data transfer).

This concept is useful for understanding basic protocols before considering real-world noisy channels.

A **noiseless channel** means:

- **No errors occur**
- **No frames are lost**
- **Data always arrives correctly**

Because there are no errors, the protocols only handle **flow control** (controlling the speed of data transfer)

Protocols for Noiseless Channels

1. Simplest Protocol

- The sender sends data continuously without waiting for any acknowledgment.
- The receiver simply accepts data as it arrives.
- No flow control or error control mechanisms.

How it works:

- Example:
Sender sends frames 1, 2, 3, 4, 5 continuously → Receiver receives them.
- If the receiver is slower, it can get overwhelmed because there is **no mechanism to slow down the sender**.

When is it used?

- Only when the sender and receiver are perfectly synchronized and the receiver can handle data at the sender's speed.

Limitations:

- Not practical for most real systems because it assumes perfect synchronization and infinite buffer at the receiver.

No ACK (Acknowledgment) means the receiver does not send any confirmation message back to the sender after receiving data.

Simple Meaning

- ACK = Acknowledgment
- It is a reply message sent by the receiver to tell the sender:
"I received the data successfully."

When there is No ACK

- The sender does not wait for confirmation.
- The sender keeps sending frames continuously.

Example

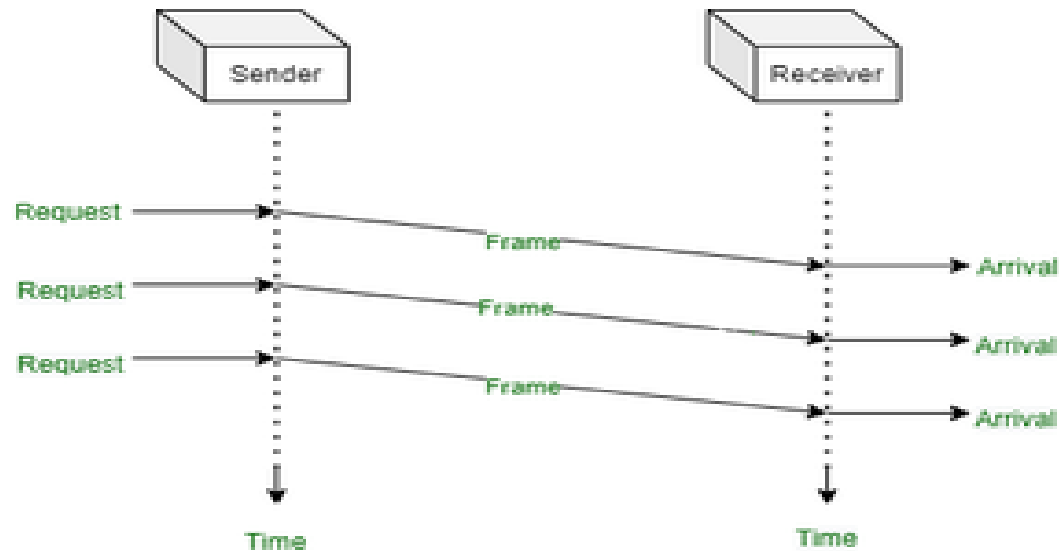
Sender sends frames:

Sender → Frame1 → Frame2 → Frame3 → Frame4 → Receiver

- Receiver gets the frames.
- Receiver does not send any reply (ACK).

Problem with No ACK

- Sender does not know if data is received correctly.
- If the receiver is slow, data may be lost or buffers may overflow.



2. Stop-and-Wait Protocol

- The sender sends **one frame at a time**.
- After sending a frame, the sender **waits for an acknowledgment (ACK)** from the receiver before sending the next frame.
- This provides basic **flow control**.

How it works:

- Sender sends frame 1 → waits → Receiver receives frame 1 and sends ACK → Sender sends frame 2 → and so on.

Advantages:

- Simple and prevents receiver buffer overflow by sending only one frame at a time.
- Works well when the channel is reliable (no errors).

Limitations:

- Inefficient if the link has high delay because the sender stays idle waiting for ACK.
- Throughput is low because only one frame is in transit at a time.

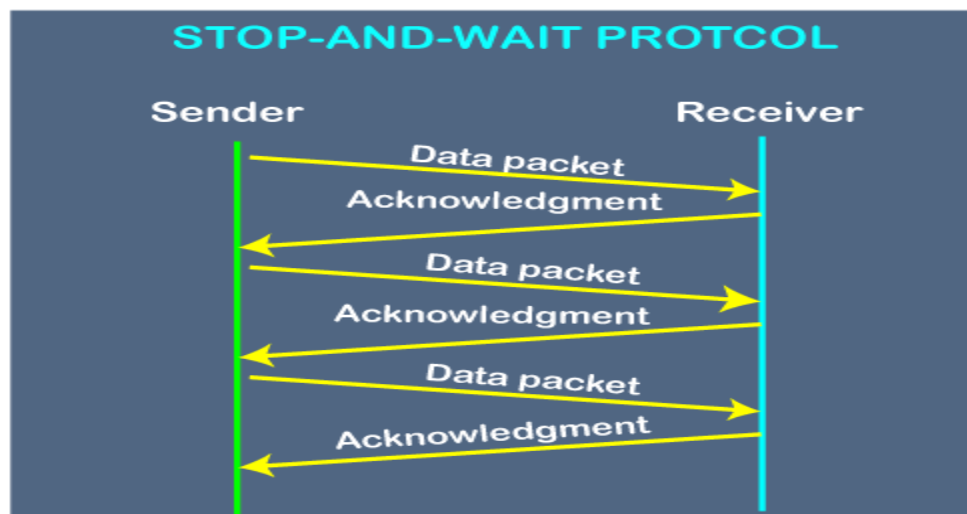
2-Noisy channel

A **noisy channel** means the communication medium may cause:

- Errors in frames
- Lost frames
- Duplicate frames

To handle these problems, the Data Link Layer uses **ARQ (Automatic Repeat Request)** protocols.

These protocols use **ACK (Acknowledgment)**, **timers**, and **retransmission**.



Noisy Channel:

In simple terms, a noise channel protocol is a set of rules that helps send data reliably over a communication line that might mess up or lose information (because of noise).

It tells the system:

1. How to check for errors in received data.
2. What to do if errors happen (fix it or resend the data).
3. When to send more data without overwhelming the receiver.

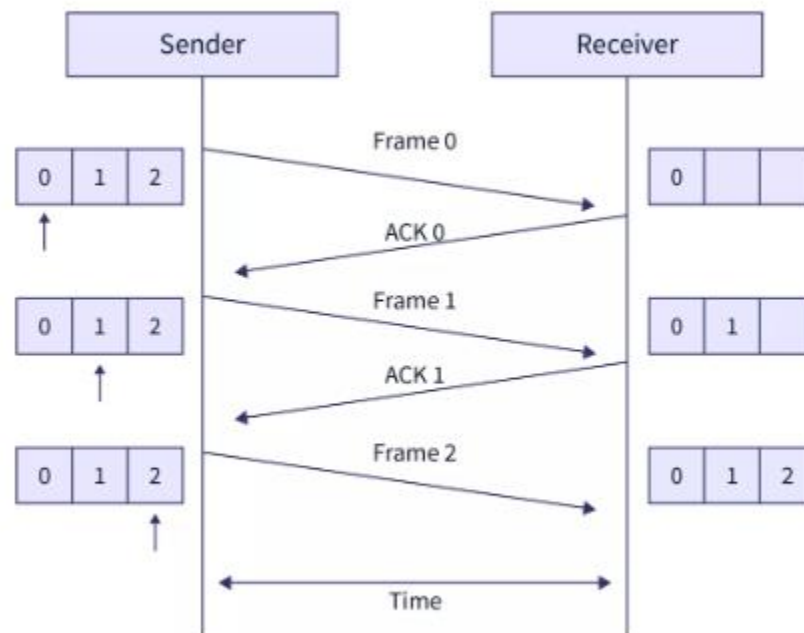
Example:

- Stop-and-Wait ARQ is a simple noise channel protocol:
 - Send 1 message → wait for confirmation → if correct, send the next; if wrong, resend.

1. Stop-and-Wait ARQ

Meaning

In **Stop-and-Wait ARQ**, the sender sends **one frame at a time** and waits for an acknowledgment before sending the next frame.



How It Works

1. Sender sends **Frame 0**.
2. Receiver checks the frame using **error detection (CRC/checksum)**.
3. If the frame is **correct**, receiver sends **ACK**.
4. If the frame is **damaged or lost**, receiver sends **NAK** (Negative Acknowledgment) or the sender retransmits after **timeout**.

Example Flow

Sender → Frame0 → Receiver

Receiver → ACK0 → Sender

Sender → Frame1 → Receiver

If error occurs:

Sender → Frame1 → (Error in channel)

No ACK received → **Timeout occurs** → Sender sends **Frame1 again**

Important Features

- Uses sequence numbers (0 and 1) to detect duplicates.
- Only one frame in the channel at a time.
- Uses ACK and timeout mechanism.

Advantages

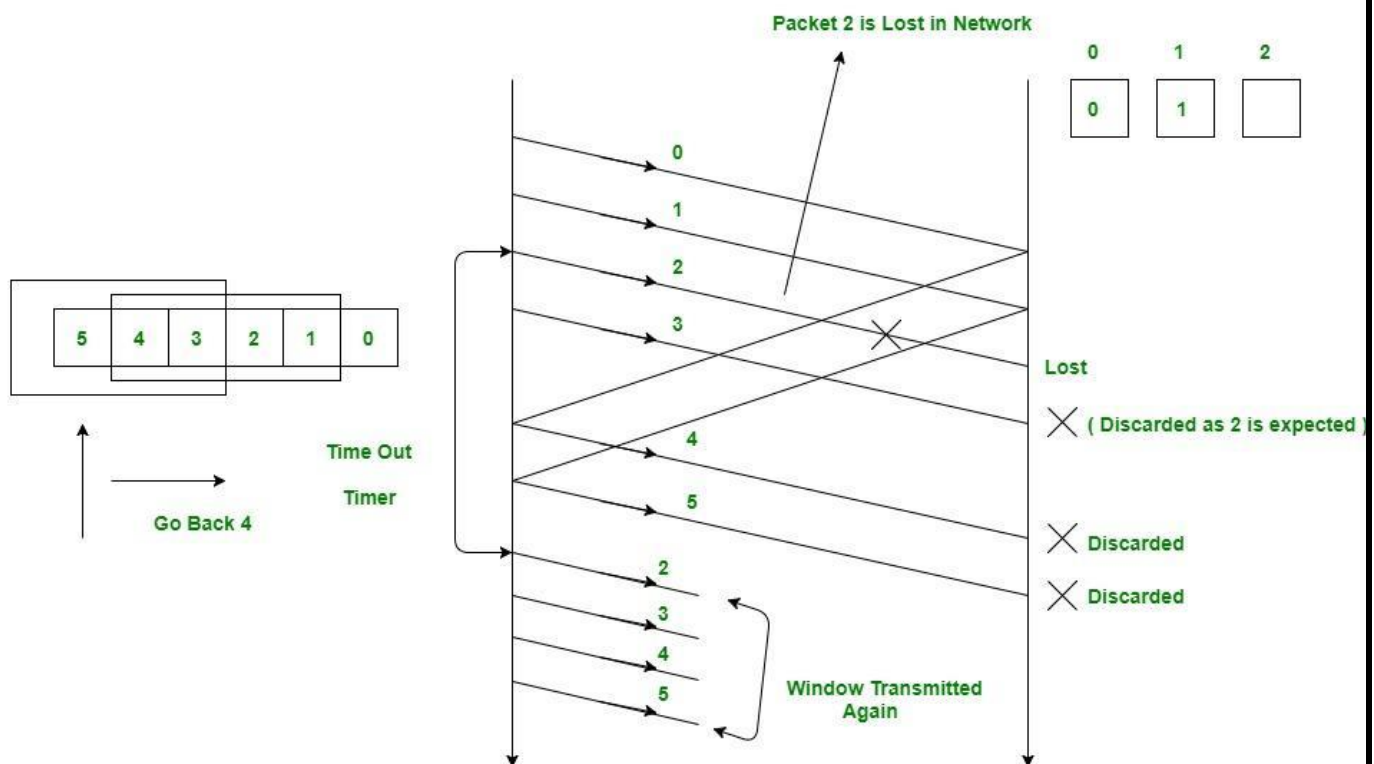
- Very simple protocol
- Easy to implement

Disadvantages

- Low efficiency for long-distance communication
- Sender stays idle while waiting for ACK

3. Go-Back-N ARQ

In Go-Back-N ARQ, the sender can send multiple frames before receiving ACK. It uses a sliding window protocol.



Sliding Window Concept

- Sender has a **window size (N)**.
- Sender can send **N frames continuously** without waiting.

Example (Window Size = 4)

Sender sends:

Frame0 → Frame1 → Frame2 → Frame3

How It Works

1. Sender sends several frames continuously.
2. Receiver checks frames in order.
3. If all frames are correct → receiver sends **ACK for last correctly received frame**.

Error Situation

Example:

Sender → Frame0 → Frame1 → Frame2 → Frame3

If **Frame2 is lost**:

- Receiver cannot accept Frame3.
- Receiver sends **ACK1** (last correct frame).

Sender then **goes back to Frame2** and retransmits:

Frame2 → Frame3 again.

Important Features

- Uses **sliding window**
- Receiver only accepts **frames in order**
- One ACK can acknowledge **multiple frames**

Advantages

- **Higher efficiency** than Stop-and-Wait
- Better use of bandwidth

Disadvantages

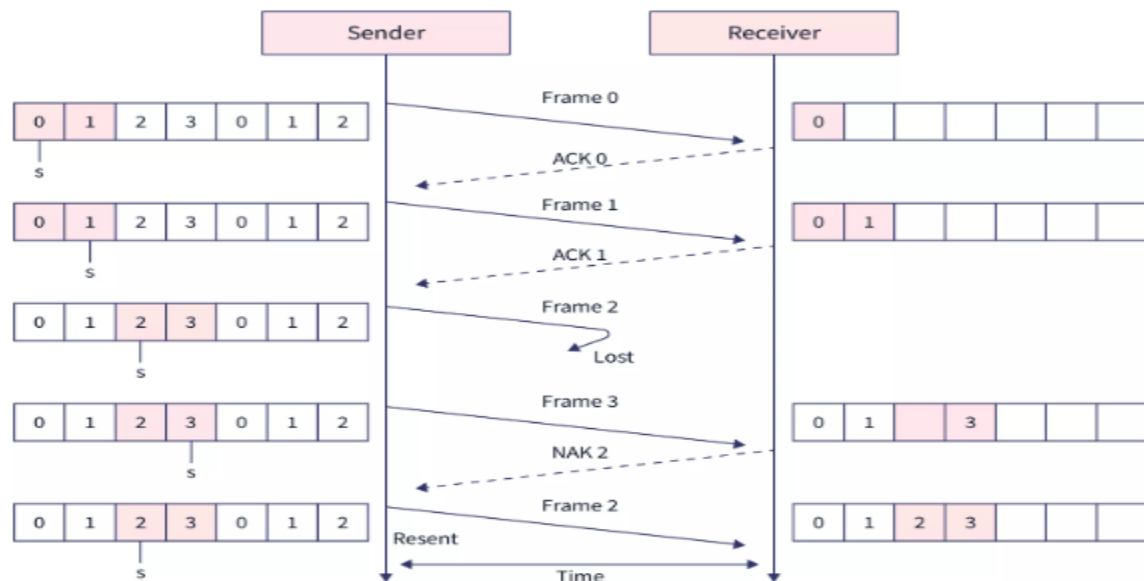
- If one frame is lost, **many frames must be retransmitted**

3. Selective Repeat ARQ

Meaning

In **Selective Repeat ARQ**, only the frames that contain errors are retransmitted.

Receiver can **store correct frames** even if some frames are missing.



How It Works

Example:

Sender → Frame0 → Frame1 → Frame2 → Frame3

If **Frame2** is lost:

- Receiver still accepts **Frame3**
- Receiver sends **NAK2**

Sender retransmits **only Frame2**.

After receiving Frame2, the receiver arranges frames in correct order.

Receiver Buffer

Receiver uses **buffer memory** to store out-of-order frames.

Example storage:

Frame0 ✓

Frame1 ✓

Frame2 ✗

Frame3 ✓ (stored in buffer)

After Frame2 arrives → all frames delivered to upper layer.

Important Features

- Uses sliding window
- Receiver accepts out-of-order frames
- Only specific frames retransmitted

Advantages

- Most efficient protocol
- Reduces unnecessary retransmission

Disadvantages

- More complex
- Requires more memory (buffers)

Multiple access protocol- ALOHA, CSMA, CSMA/CA and CSMA/CD Data

Link Layer :-

The [data link layer](#) is used in a computer network to transmit the data between two devices or nodes. It divides the layer into parts such as data link control and the multiple access resolution/protocol. The upper layer has the responsibility to flow control and the error control in the data link layer, and hence it is termed as logical of data link control. Whereas the lower sub-layer is used to handle and reduce the collision or multiple access on a channel. Hence it is termed as [media access control](#) or the multiple access resolutions.

Data Link Control

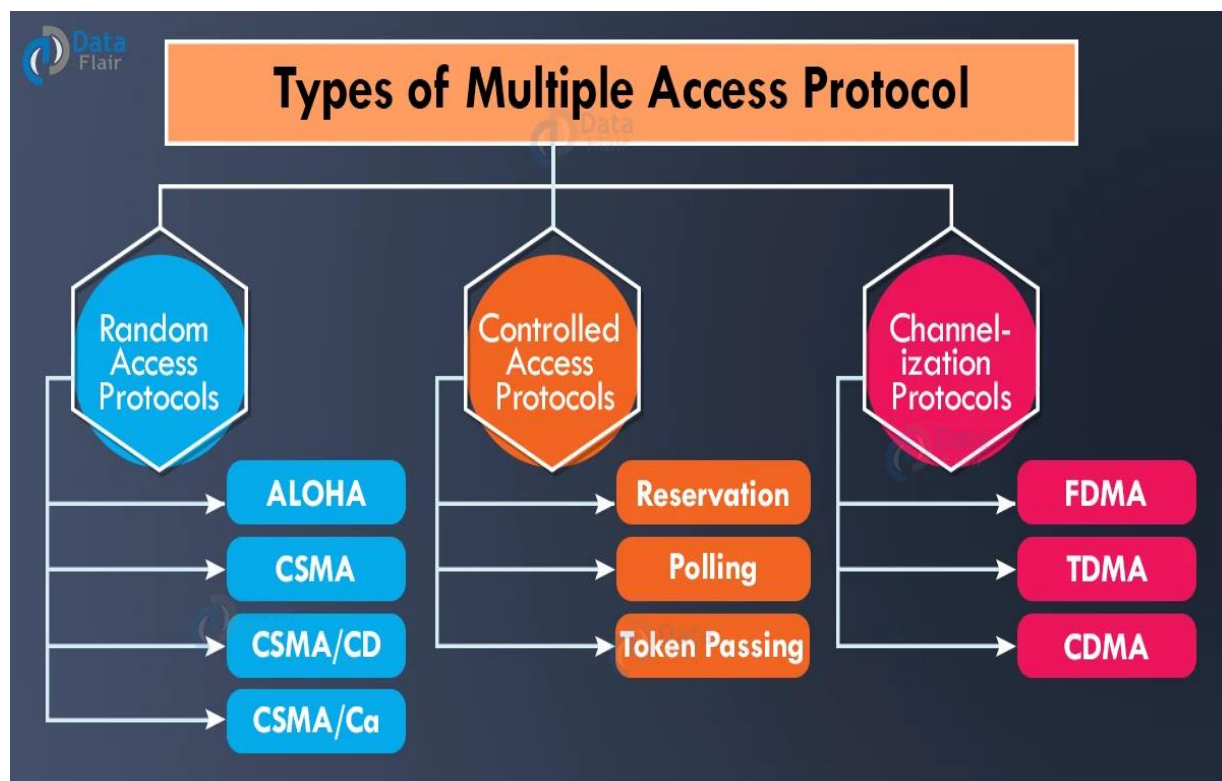
A data link control is a reliable channel for transmitting data over a dedicated link using various techniques such as framing, error control and flow control of data packets in the computer network.

What is a multiple access protocol?

When a sender and receiver have a dedicated link to transmit data packets, the data link control is enough to handle the channel. Suppose there is no dedicated path to communicate or transfer the data between two devices. In that case, multiple stations access the channel and simultaneously transmits the data over the channel. It may create collision and cross talk. Hence, the multiple access protocol is required to reduce the collision and avoid crosstalk between the channels.

For example, suppose that there is a classroom full of students. When a teacher asks a question, all the students (small channels) in the class start answering the question at the same time (transferring the data simultaneously). All the students respond at the same time due to which data is overlap or data lost. Therefore it is the responsibility of a teacher (multiple access protocol) to manage the students and make them one answer.

Following are the types of multiple access protocol that is subdivided into the different process as:



A-Random Access Protocol

In this protocol, all the station has the equal priority to send the data over a channel. In random access protocol, one or more stations cannot depend on another station nor any station control another station. Depending on the channel's state (idle or busy), each station transmits the data frame. However, if more than one station sends the data over a channel, there may be a collision or data conflict. Due to the collision, the data frame packets may be lost or changed. And hence, it does not receive by the receiver end.

Following are the different methods of random-access protocols for broadcasting frames on the channel.

1. Aloha
2. CSMA
3. CSMA/CD
4. CSMA/CA

1-ALOHA Random Access Protocol

ALOHA is the earliest and simplest random-access protocol developed for satellite communication.

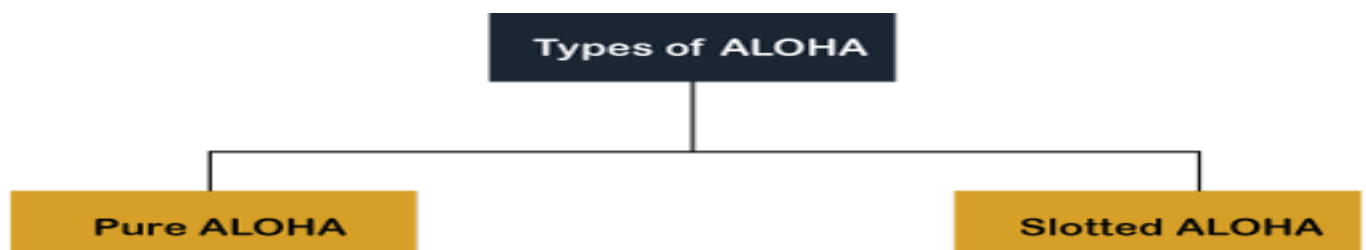
It is designed for wireless LAN (Local Area Network) but can also be used in a shared medium to transmit data. Using this method, any station can transmit data across a network simultaneously when a data frameset is available for transmission.

Working Principle

1. A device sends data whenever it has data to send.
2. If two devices send data at the same time, a collision occurs.
3. The receiver cannot understand the data.
4. The sender waits for a random time and then retransmits the data.

Aloha Rules

1. Any station can transmit data to a channel at any time.
2. It does not require any carrier sensing.
3. Collision and data frames may be lost during the transmission of data through multiple stations.
4. Acknowledgment of the frames exists in Aloha. Hence, there is no collision detection.
5. It requires retransmission of data after some random amount of time.



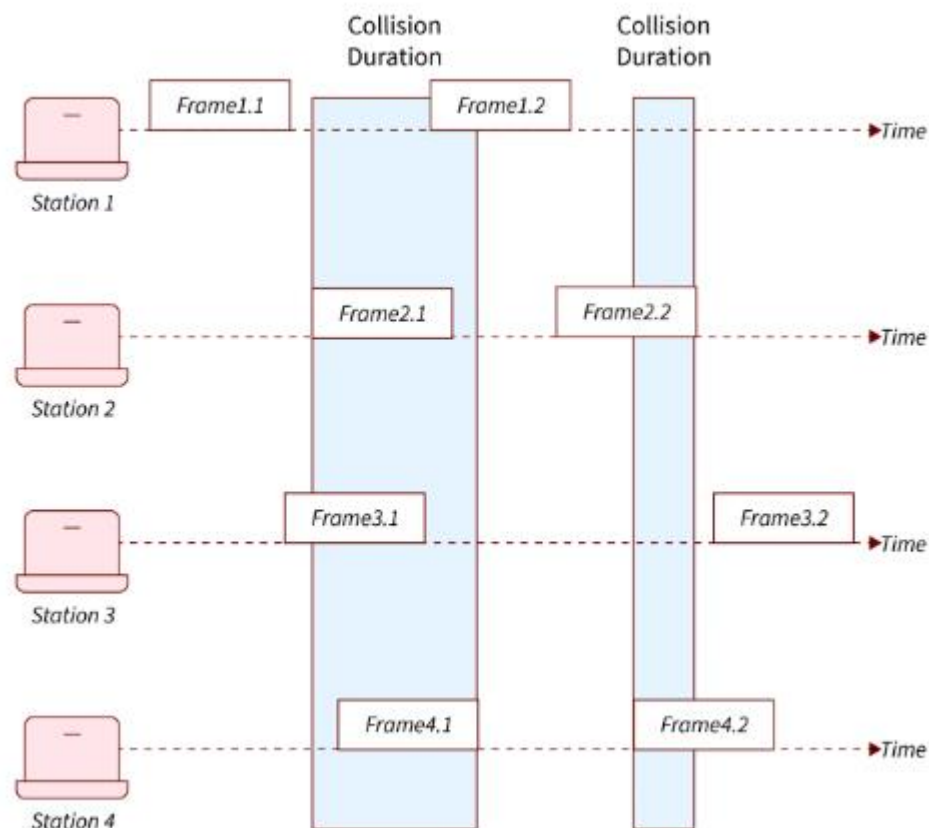
1-Pure Aloha:-

Whenever data is available for sending over a channel at stations, we use Pure Aloha. In pure Aloha, when each station transmits data to a channel without checking whether the channel is idle or not, the chances of collision may occur, and the data frame can be lost.

When any station transmits the data frame to a channel, the pure Aloha waits for the receiver's acknowledgment. If it does not acknowledge the receiver end within the specified time, the station waits for a random amount of time, called the backoff time (T_b). And the station may assume the frame has been lost or destroyed.

Therefore, it retransmits the frame until all the data are successfully transmitted to the receiver.

1. The total vulnerable time of pure Aloha is $2 * T_{fr}$.
2. Maximum throughput occurs when $G = 1/2$ that is 18.4%.
3. Successful transmission of data frame is $S = G * e^{-2G}$.



The diagram shows how collisions occur in the ALOHA protocol when multiple stations send data over the same channel.

- There are four stations: Station 1, Station 2, Station 3, and Station 4.
- Each station sends frames (data packets) such as Frame1.1, Frame2.1, Frame3.1, etc.
- In ALOHA, stations transmit data whenever they have data, without checking the channel.

Collision

- When two or more stations transmit at the same time, their frames overlap.
- This overlapping causes a collision, shown by the blue shaded area (Collision Duration) in the diagram.
- During this time, the frames are corrupted and lost.

Retransmission

- After a collision, the stations wait for a random time.
- Then they retransmit the frames (e.g., Frame1.2, Frame2.2, etc.).

Conclusion

The diagram illustrates that ALOHA has a high probability of collisions because stations send data without sensing the channel.

2-Slotted Aloha

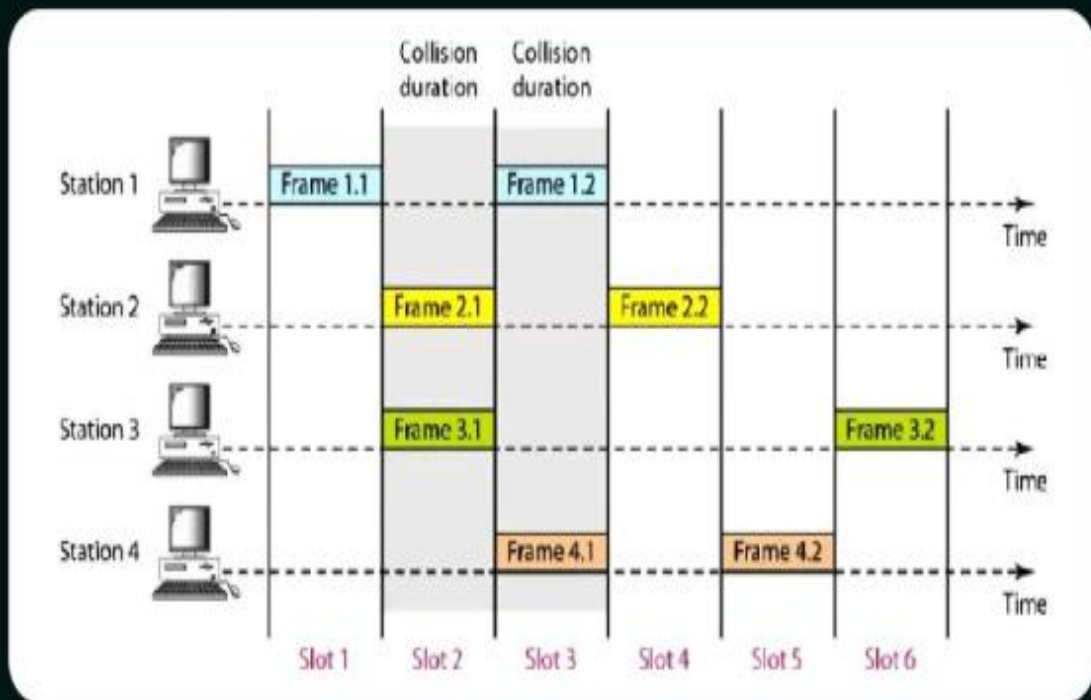
The slotted Aloha is designed to overcome the pure Aloha's efficiency because pure Aloha has a very high possibility of frame hitting. In slotted Aloha, the shared channel is divided into a fixed time interval Called slots.

So that, if a station wants to send a frame to a shared channel, the frame can only be sent at the beginning of the slot, and only one frame is allowed to be sent to each slot. And if the stations are unable to send data to the beginning of the slot, the station will have to wait until the beginning of the slot for the next time.

However, the possibility of a collision remains when trying to send a frame at the beginning of two or more station time slot.

1. Maximum throughput occurs in the slotted Aloha when $G = 1$ that is 37%.
2. The probability of successfully transmitting the data frame in the slotted Aloha is $S = G * e^{-2G}$.
3. The total vulnerable time required in slotted Aloha is T_{fr} .

Slotted Aloha



The diagram shows how Slotted ALOHA works.

- In Slotted ALOHA, time is divided into equal slots (Slot 1, Slot 2, Slot 3, etc.).
- A station can transmit data only at the beginning of a time slot.
- Multiple stations (Station 1, 2, 3, 4) send frames such as Frame1.1, Frame2.1, etc.

Collision

- If two or more stations send data in the same slot, their frames collide.
- The grey area in the diagram shows the collision duration.

Successful Transmission

- If only one station transmits in a slot, the frame is successfully received.

Result

Slotted ALOHA reduces collisions compared to Pure ALOHA, so it has better efficiency (about 36%).

2-CSMA (Carrier Sense Multiple Access)

CSMA is a Random Access Protocol used in computer networks to reduce collisions.

Working Principle

- Before sending data, a station checks the channel to see if it is busy or free.
- If the channel is free, the station transmits the data.
- If the channel is busy, the station waits until the channel becomes free.

Steps

1. A station wants to send data.
2. It senses (listens to) the channel.
3. If the channel is idle → transmit data.
4. If the channel is busy → wait and try again.

Collision

Even after sensing the channel, collision may still occur because two stations may start transmitting at the same time due to propagation delay.

Types of CSMA

1. 1-Persistent CSMA – Station sends data immediately when the channel becomes free.
2. Non-Persistent CSMA – Station waits for a random time before checking the channel again.
3. p-Persistent CSMA – Station sends data with probability p when the channel is free.

Advantages

- Reduces collisions compared to ALOHA
- Better channel utilization

Disadvantages

- Collisions can still occur
- Delay may increase due to waiting

1-Persistent:

In the 1-Persistent mode of CSMA that defines each node, first sense the shared channel and if the channel is idle, it immediately sends the data. Else it must wait and keep track of the status of the channel to be idle and broadcast the frame unconditionally as soon as the channel is idle.

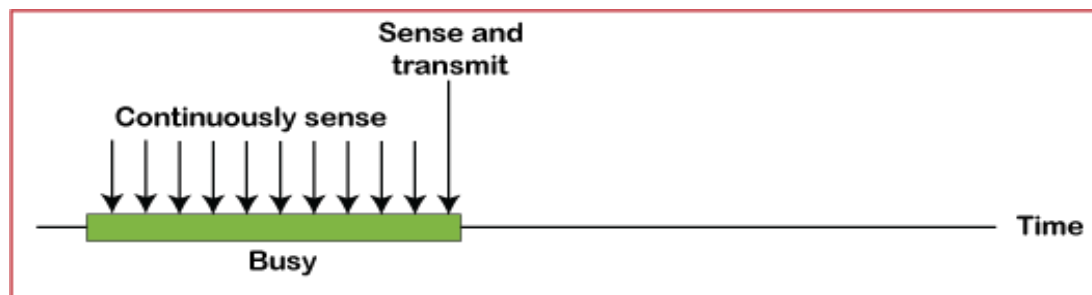
Non-Persistent:

It is the access mode of CSMA that defines before transmitting the data, each node must sense the channel, and if the channel is inactive, it immediately sends the data. Otherwise, the station must wait for a random time (not continuously), and when the channel is found to be idle, it transmits the frames.

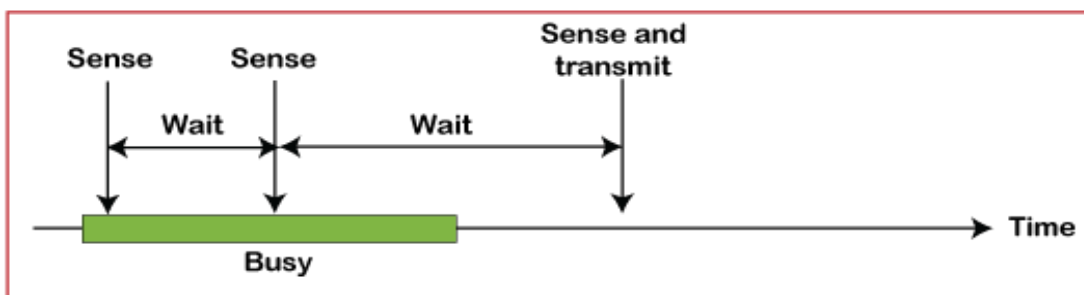
P-Persistent:

It is the combination of 1-Persistent and Non-persistent modes. The P-Persistent mode defines that each node senses the channel, and if the channel is inactive, it sends a frame with a P probability. If the data is not transmitted, it waits for a ($q = 1-p$ probability) random time and resumes the frame with the next time slot.

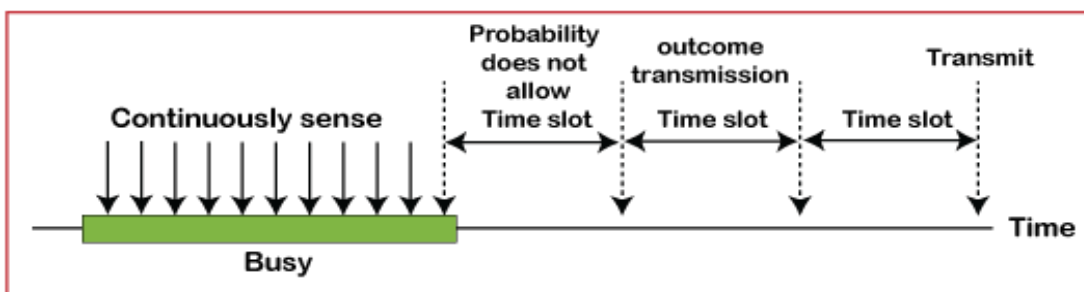
O-Persistent: It is an O-persistent method that defines the superiority of the station before the transmission of the frame on the shared channel. If it is found that the channel is inactive, each station waits for its turn to retransmit the data.



a. 1-persistent



b. Nonpersistent



c. p-persistent

3-CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

CSMA/CD is a random-access protocol used mainly in wired Ethernet networks to control how devices share a communication channel.

Working Principle

In CSMA/CD, a station listens to the channel before sending data and detects collisions while transmitting.

Steps of CSMA/CD

1. Carrier Sense

The station first checks the channel to see if it is free or busy.

2. Transmission

If the channel is idle, the station starts transmitting data.

3. Collision Detection

While transmitting, the station monitors the channel to detect if another station is transmitting at the same time.

4. Collision Occurs

If a collision is detected, the station:

- Stops transmission immediately
- Sends a jam signal to inform other stations

5. Backoff and Retransmission

The station waits for a random backoff time and then retransmits the data.

Advantages

- Detects collisions quickly
- Efficient for wired networks

Disadvantages

- Not suitable for wireless networks
- Requires collision detection hardware

Example

Used in Ethernet LAN networks.

4-CSMA/CA is a random access protocol used mainly in wireless networks (Wi-Fi) to reduce collisions.

Working Principle

In wireless networks, it is difficult to detect collisions, so CSMA/CA tries to avoid collisions before transmission.

Steps of CSMA/CA

1. Carrier Sense

The station checks the channel to see if it is free or busy.

2. Wait Time (IFS)

If the channel is free, the station waits for a short time called Interframe Space (IFS).

3. Backoff Timer

The station starts a random backoff timer.

When the timer reaches zero, the station transmits the data.

4. RTS/CTS Mechanism (Optional)

- Sender sends RTS (Request To Send).
- Receiver replies with CTS (Clear To Send).
- Other stations wait until transmission is complete.

5. Data Transmission

After permission, the station sends the data.

Advantages

- Reduces the chance of collisions
- Suitable for wireless networks

Disadvantages

- Extra control messages cause more delay

Example

Used in Wi-Fi and Wireless LAN networks.

Here is a clear, concise comparison between **CSMA/CA** and **CSMA/CD**:

Feature	CSMA/CD (Collision Detection)	CSMA/CA (Collision Avoidance)
Used In	Wired Ethernet networks	Wireless networks (Wi-Fi)
Collision Handling	Detects collision during transmission	Tries to avoid collision before transmission
Collision Detection	Yes, devices monitor channel to detect collision	No collision detection, collision is avoided
Mechanism	Stops transmission immediately on collision and retransmits after backoff	Uses waiting times and RTS/CTS control messages before sending data
Channel Sensing	Listens before and during transmission	Listens before transmission
Efficiency	High in wired networks	Good for wireless but slightly lower efficiency
Delay	Less delay due to immediate collision detection	More delay due to backoff and control messages
Suitability	Not suitable for wireless networks due to hidden node problem	Suitable for wireless where collision detection is hard
Example Protocols	Ethernet (IEEE 802.3)	Wi-Fi (IEEE 802.11)

B-Controlled Access Protocols:

Controlled Access Protocols are multiple access methods where stations take turns to use the channel, so collisions are avoided. Unlike random access protocols (like ALOHA or CSMA), here only one station can transmit at a time, following a predefined rule.

These protocols are more efficient than random access for heavy traffic network

Types of Controlled Access Protocols

1. Polling
 2. Token Passing
 3. Reservation
-

1. Polling

- A central controller (master station) gives permission to each station in turn.
- Only the station with permission can send data.
- After transmitting, the station reports back to the master, and the master polls the next station.

Advantages:

- No collisions
- Fair allocation of channel

Disadvantages:

- If the master fails, the system stops
- Extra overhead from polling

Example:

Used in IBM Token Ring networks (with centralized polling concept).

2. Token Passing

- A special control frame called a token circulates the network.
- Only the station holding the token can transmit data.
- After transmission, the station passes the token to the next station.

Advantages:

- No collisions
- Efficient for high-speed networks

Disadvantages:

- Token lost → network halt
- Slight delay for token circulation

Example:

- Token Ring networks
 - FDDI (Fiber Distributed Data Interface)
-

3. Reservation

- Stations reserve the channel before transmitting.
- Time is divided into slots, and stations signal their intention in advance.
- Only reserved slots are used for transmission.

Advantages:

- Collision-free transmission
- Works well under heavy traffic

Disadvantages:

- Extra overhead for reservations
- Complex to implement

Example:

- Satellite communication systems
-

C- Channelization Protocol

A Channelization Protocol is a network method used to divide a single communication channel into smaller parts, allowing multiple users to transmit data simultaneously without collisions. Instead of competing for the channel, each user gets a dedicated portion.

1-FDMA (Frequency Division Multiple Access)-

FDMA is a channelization protocol in which the total available bandwidth of a communication channel is divided into separate frequency bands, and each user is assigned a unique frequency band for communication.

This allows multiple users to transmit simultaneously without interference, as each one uses a different frequency.

How FDMA Works

1. Total Bandwidth Division
 - Suppose a channel has a bandwidth of 30 MHz.
 - FDMA divides it into 10 smaller bands, each 3 MHz.

2. Frequency Assignment

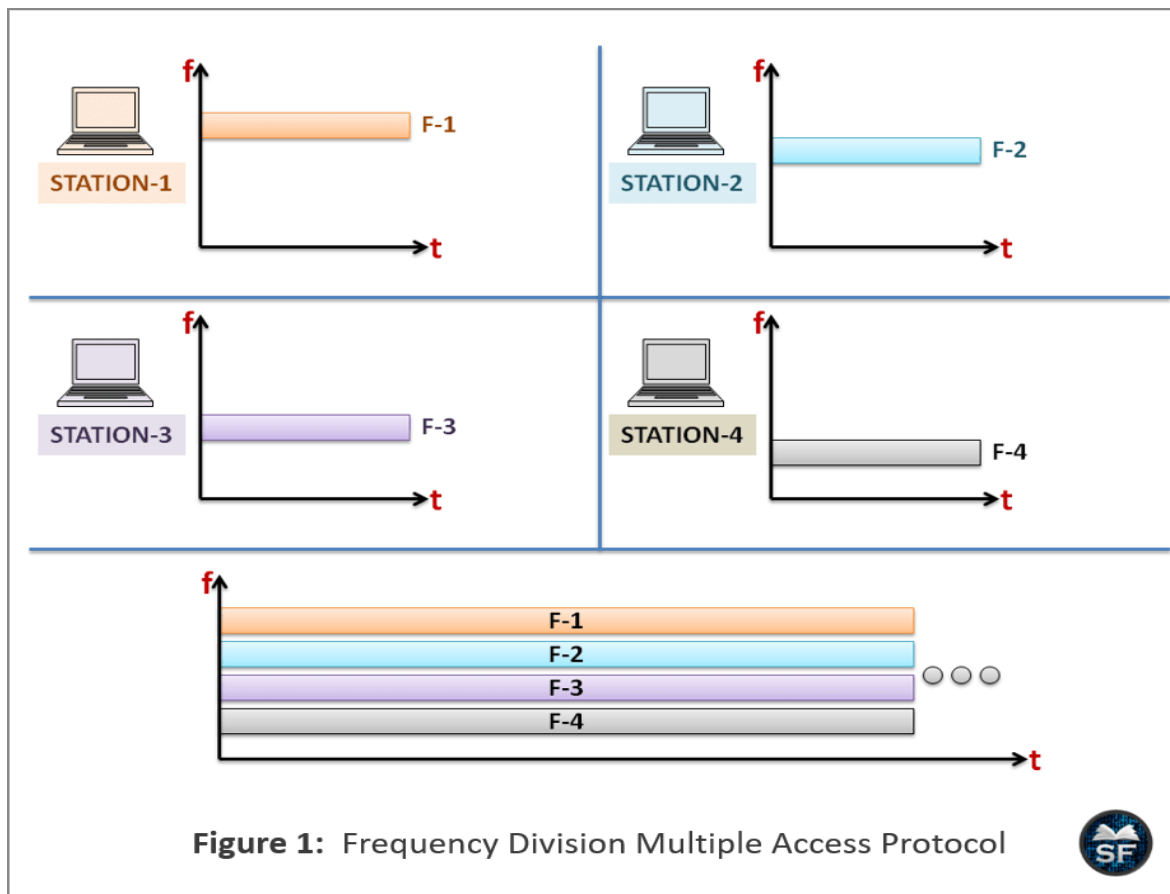
- Each user is assigned a dedicated frequency band.
-
- Example:
 - User 1 → 0–3 MHz
 - User 2 → 3–6 MHz
 - User 3 → 6–9 MHz, and so on.

3. Simultaneous Transmission

- Users can transmit at the same time because their signals occupy different frequencies.
- There is no collision, as signals do not overlap in the frequency domain.

4. Guard Bands

- Small unused frequency bands, called guard bands, are placed between user bands.
- These prevent interference between adjacent users.



Advantages of FDMA

1. Simultaneous Communication – All users can send data at the same time.
 2. Simple Implementation – Easy to design transmitters and receivers using filters for frequency separation.
 3. Continuous Transmission – Users don't have to wait for a time slot (unlike TDMA).
-

Disadvantages of FDMA

1. Inefficient Bandwidth Usage – Guard bands waste some portion of bandwidth.
 2. Limited Users – The number of users is limited by the total available bandwidth.
 3. No Flexibility – If some users are idle, their frequency band is wasted.
 4. Complex Filtering – Requires precise frequency filters to separate signals.
-

Applications of FDMA

- Analog Mobile Telephony – Early 1G mobile networks used FDMA.
 - Satellite Communication – Each satellite channel gets a frequency band.
 - Radio Broadcasting – Different radio stations use different frequencies.
-

(Simple Conceptual Diagram)

Total Channel Bandwidth

|-----|
|User1|GB|User2|GB|User3|GB|User4|

- GB = Guard Band
 - Each user has a dedicated frequency slice.
-

2-TDMA (Time Division Multiple Access)

TDMA is a channelization protocol in which the total available bandwidth is shared among multiple users in the time domain.

Instead of giving each user a separate frequency (like FDMA), all users share the same frequency, but each user can transmit only during their assigned time slot.

How TDMA Works

1. Time Division

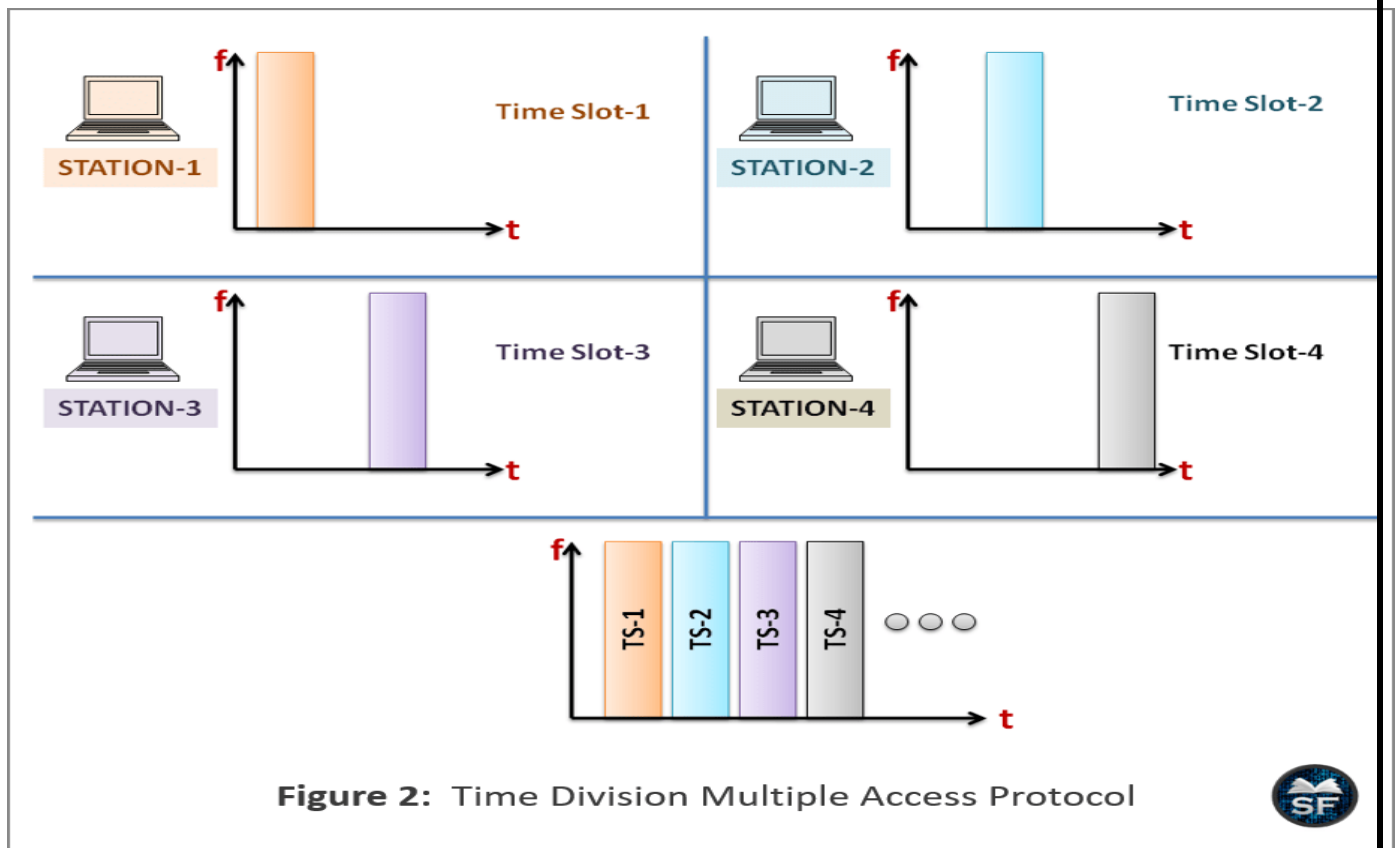
- The communication channel is divided into **repeating time slots**.
- Each time slot is assigned to a different user.

2. Transmission

- During its time slot, a user can transmit data.
- Other users must wait for their own time slot.
- After the last user, the cycle repeats.

3. Synchronization

- Precise timing is required so that users transmit only during their slot.
- A master clock or synchronization signal is used to align transmissions.



Advantages of TDMA

1. Efficient Use of Bandwidth – Unlike FDMA, guard bands are not required for each user.
2. Supports Multiple Users – Many users can share the same frequency.
3. Power Efficient for Mobile Devices – Devices transmit only during their slot, saving energy.

Disadvantages of TDMA

1. Synchronization Required – Devices must be precisely synchronized, which adds complexity.
2. Delay – A user must wait for their time slot to transmit, which can add latency.
3. Fixed Slot Wastage – If a user has no data to send during its slot, that slot is wasted.

Applications of TDMA

- 2G Mobile Networks – GSM (Global System for Mobile Communications) uses TDMA.
- Satellite Communication – Multiple ground stations share a satellite channel.
- Digital Voice Transmission – Efficient for voice because short time slots match voice bursts.

(Simple Conceptual Diagram)

Time → |Slot1|Slot2|Slot3|Slot4|Slot1|Slot2|Slot3|Slot4|

User → |U1 |U2 |U3 |U4 |U1 |U2 |U3 |U4 |

- Each user transmits only during its assigned time slot.
- After all users transmit, the cycle repeats.

3-CDMA (Code Division Multiple Access)

CDMA is a channelization protocol in which all users share the same frequency and time, but each user is assigned a unique code to encode their signal.

The receiver uses this code to extract the intended signal, allowing simultaneous communication without collisions.

Unlike FDMA (frequency) or TDMA (time), CDMA separates users using codes, not frequency bands or time slots.

How CDMA Works

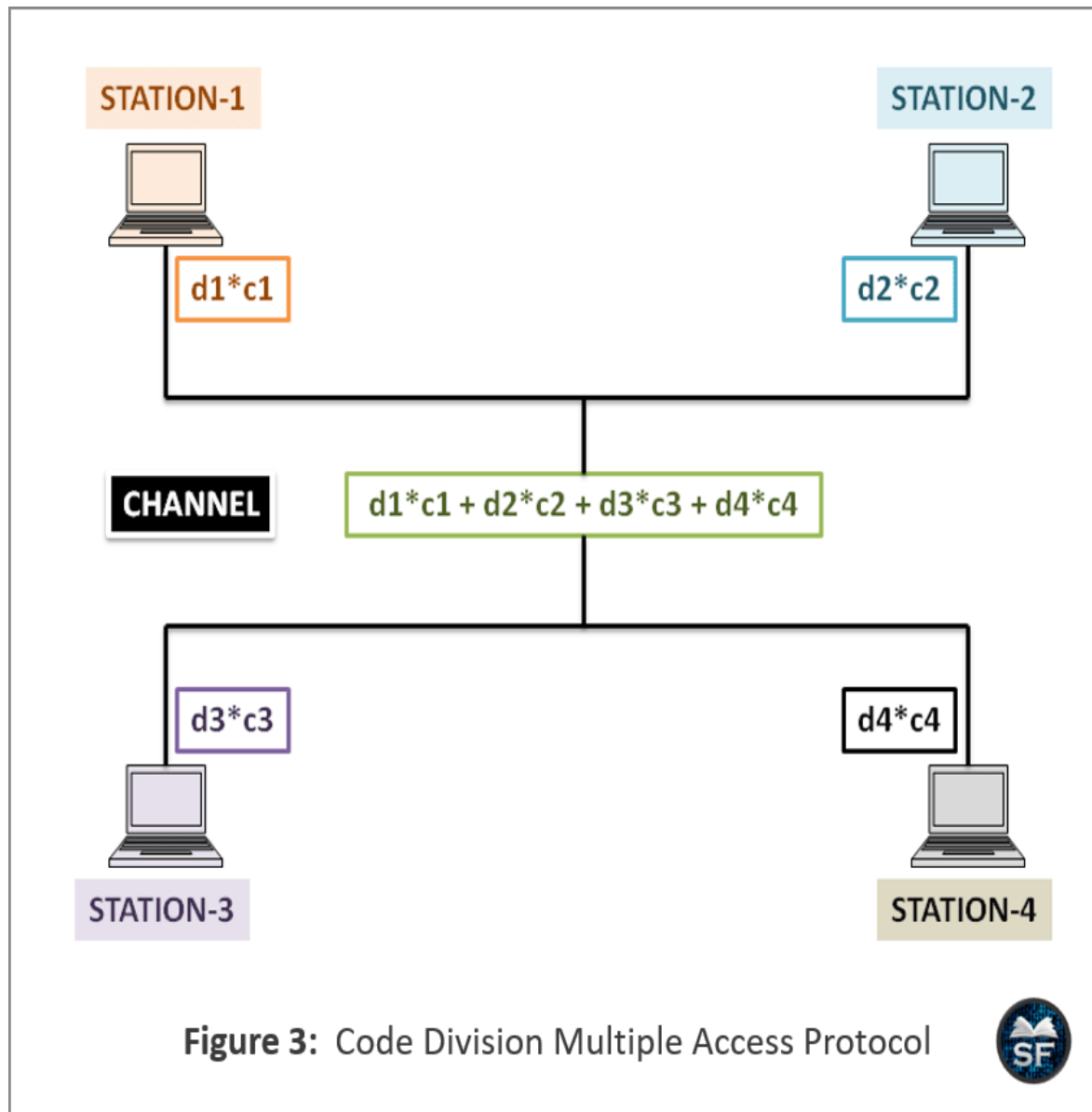
1. Unique Code Assignment
 - Each user is assigned a pseudo-random code.
 - This code spreads the signal across a wide frequency band (spread spectrum).
2. Encoding
 - The user's data signal is multiplied by their unique code.
 - This "spreads" the signal over a wide frequency range.

3. Transmission

- Multiple users transmit simultaneously over the same frequency band.

4. Decoding

- The receiver knows the user's unique code.
- It uses the code to correlate and extract the intended signal while treating other signals as noise.



Advantages of CDMA

1. High Efficiency – Many users can share the same frequency at the same time.
2. Resistance to Interference – Spread spectrum makes it robust against noise and interference.
3. Security – Unique codes make eavesdropping harder.

4. Flexible Bandwidth Usage – Idle users don't waste resources; active users can still transmit.

Disadvantages of CDMA

1. Complex Encoding/Decoding – Requires more processing power for code correlation.
 2. Near-Far Problem – Stronger signals can overpower weaker ones if power control is not used.
 3. Code Management Needed – Codes must be carefully assigned to avoid interference.
-

Applications of CDMA

- 3G Mobile Networks – CDMA2000, WCDMA (used in UMTS)
 - Satellite Communication – Multiple ground stations share same channel.
 - Military Communications – Spread spectrum makes it secure and resistant to jamming.
-

(Simple Conceptual Diagram)

Frequency Band → |----- Entire Band-----|

Users:

U1 → Encoded with Code1

U2 → Encoded with Code2

U3 → Encoded with Code3

- All users transmit **over the same frequency at the same time**.
- Each user's signal is **extracted using their unique code**.

Key

- FDMA → divides frequency. Simple but limited.
- TDMA → divides time. Efficient but needs synchronization.
- CDMA → divides code. Complex but supports many simultaneous users with high security.

Here's a clear comparison of FDMA, TDMA, and CDMA, showing the differences side by side:

Feature	FDMA (Frequency Division)	TDMA (Time Division)	CDMA (Code Division)
How channel is divided	Frequency bands	Time slots	Unique codes
Simultaneous users	Yes, on different frequencies	Yes, but in different time slots	Yes, all users at the same time & frequency
Synchronizatio n needed	Low	High – precise timing required	Moderate – code synchronization required
Collision possibility	Very low (separate frequencies)	Low (time slots prevent overlap)	Very low (codes separate signals)
Efficiency	Wasted bandwidth if users idle; guard bands needed	Wasted if user has no data in time slot	Very efficient; idle users don't waste resources
Complexity	Low (simple filtering)	Medium (time slot managemen t)	High (encoding/decodin g)
Capacity	Limited by total bandwidth	Moderate	High – many users can share same frequency
Examples	1G mobile, radio broadcastin g	GSM (2G) mobile networks	CDMA2000, WCDMA (3G), military comms